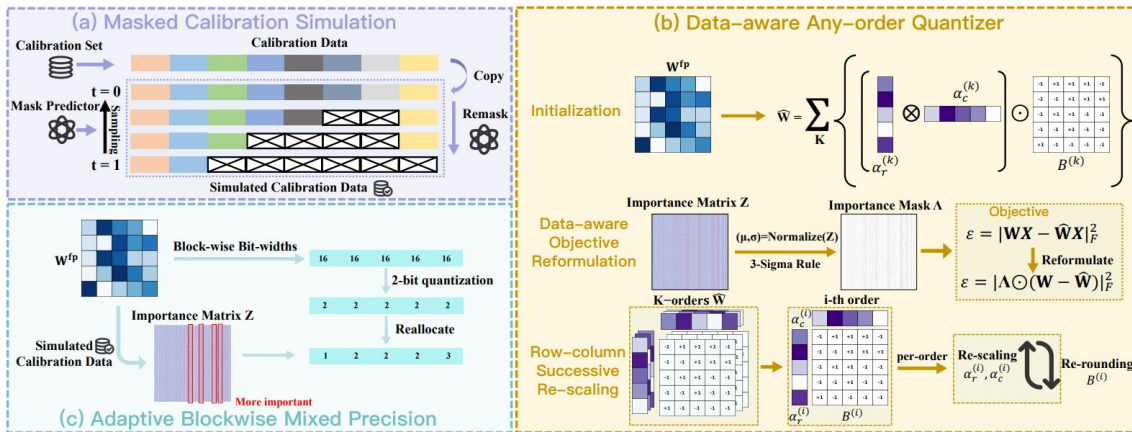


Literature Review: DLM Quantization + SING

Ahmad Fraij

Quant-dLLM

- Quant-dllm is a weight only calibrated 2-bit PTQ technique
- This framework resolves two main issues in DLMs:
 - Changing distributions of activations over timesteps
 - Accumulation of quantization errors across denoising steps



Quant-dLLM MCS

Algorithm 1 MCS: Masked Calibration Simulation

func MCS($\mathcal{X}, T, \gamma, \alpha(t)$)

Input: $\mathcal{X}$ — full sequences; T — total timesteps;

γ — prefix ratio; $\alpha(t)$ — visibility schedule

Output: Masked calibration set $\mathcal{D}_{\text{calib}}$

1: $\mathcal{D}_{\text{calib}} \leftarrow \emptyset$

2: **for all** $x \in \mathcal{X}$ **do**

3: $L \leftarrow$ length of x

4: $\mathcal{P} \leftarrow \{1, \dots, \lfloor \gamma \cdot L \rfloor\}$

5: **for** $t = 1$ to T **do**

6: $\alpha \leftarrow \alpha(t)$

7: **for** $i = 1$ to L **do**

8: $r_i \leftarrow 1$ if $i \in \mathcal{P}$ else Bernoulli(α)

9: $\tilde{x}_i(t) \leftarrow x_i$ if $r_i = 1$ else [MASK]

10: **end for**

11: $\mathcal{D}_{\text{calib}} \leftarrow \mathcal{D}_{\text{calib}} \cup \{\tilde{x}(t)\}$

12: **end for**

13: **end for**

14: **return** $\mathcal{D}_{\text{calib}}$

Quant-dLLM DAQ

- Minimize error on layer's output instead of minimizing the weight reconstruction error
- Quantization error is not uniformly distributed and focus on small subset of weights in DLMs
- Calculate importance scores of weights using calibration data

$$\mathcal{L}_2 = \|\mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X}\|_F^2$$

$$\mathbf{S}_{\text{MCS}} = \mathbb{E}_{t \sim \pi} \left[\sum_b \tilde{\mathbf{X}}_b(t) \tilde{\mathbf{X}}_b(t)^\top \right] \approx \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \sum_b \tilde{\mathbf{X}}_b(t) \tilde{\mathbf{X}}_b(t)^\top$$

$$\widehat{\mathcal{L}}_{\Lambda}(\boldsymbol{\alpha}_r, \boldsymbol{\alpha}_c) = \left\| \boldsymbol{\Lambda} \odot (\mathbf{W} - (\boldsymbol{\alpha}_r \boldsymbol{\alpha}_c^\top) \odot \mathbf{B}) \right\|_F^2.$$

Quant-dLLM DAQ

Algorithm 2 DAQ: Data-aware any-order Quantizer

```

func DAQ(W, Z, K, T, λ)
Input W, Z ∈ ℝn×m, K, T ∈ ℕ, λ > 1
Output
     $\hat{\mathbf{W}} = \sum_{k=1}^K (\alpha_r^{(k)} \alpha_c^{(k)\top}) \odot \mathbf{B}^{(k)}$ 
    1:  $\Lambda := \text{build\_importance\_mask}(\mathbf{Z}, \lambda)$ 
    2:  $\hat{\mathbf{W}} := \mathbf{0}_{n \times m}$ 
    3: for k = 1, 2, ..., K do
    4:    $\mathbf{R} \leftarrow \mathbf{W} - \hat{\mathbf{W}}$ 
    5:    $\alpha_r^{(k)}, \alpha_c^{(k)}, \mathbf{B}^{(k)} \leftarrow \text{binary\_rc.init}(\mathbf{R})$ 
    6:    $\mathbf{S}^{(k)} \leftarrow \alpha_r^{(k)} (\alpha_c^{(k)})^\top$ 
    7:    $\hat{\mathbf{W}} \leftarrow \hat{\mathbf{W}} + \mathbf{S}^{(k)} \odot \mathbf{B}^{(k)}$ 
    8: end for
    9: for t = 1, 2, ..., T do
    10:  for k = 1, 2, ..., K do
    11:     $\hat{\mathbf{W}}_{\setminus k} \leftarrow \sum_{q \neq k} (\mathbf{S}^{(q)} \odot \mathbf{B}^{(q)})$ 
    12:     $\mathbf{R}^{(k)} \leftarrow \mathbf{W} - \hat{\mathbf{W}}_{\setminus k}$ 
    13:     $\alpha_r^{(k)} \leftarrow \text{update\_}\alpha_r(\mathbf{R}^{(k)}, \mathbf{B}^{(k)}, \alpha_c^{(k)}, \Lambda)$ 
    14:     $\alpha_c^{(k)} \leftarrow \text{update\_}\alpha_c(\mathbf{R}^{(k)}, \mathbf{B}^{(k)}, \alpha_r^{(k)}, \Lambda)$ 
    15:     $\mathbf{S}^{(k)} \leftarrow \alpha_r^{(k)} (\alpha_c^{(k)})^\top$ 
    16:  end for
    17:   $\{\mathbf{B}^{(k)}\}_{k=1}^K \leftarrow \text{update\_B}(\mathbf{W}, \{\alpha_r^{(k)}\}_{k=1}^K, \{\alpha_c^{(k)}\}_{k=1}^K, \{\mathbf{B}^{(k)}\}_{k=1}^K)$ 
    18:   $\hat{\mathbf{W}} \leftarrow \sum_{k=1}^K \mathbf{S}^{(k)} \odot \mathbf{B}^{(k)}$ 
    19: end for
    20: return  $\hat{\mathbf{W}}$ 

func binary_rc.init(X)
  1:  $\alpha_r \leftarrow \frac{1}{m} \|\mathbf{X}\| \mathbf{1}_m$ 
  2:  $\alpha_c \leftarrow \frac{1}{n} \|\mathbf{X}\|^\top \text{diag}(\alpha_r)^{-1} \mathbf{1}_n$ 
  3:  $\mathbf{B} \leftarrow \text{sign}(\mathbf{X})$ 
  4: return  $\alpha_r, \alpha_c, \mathbf{B}$ 

func update_αr(X, B, αc, Λ)
  1:  $\mathbf{u} \leftarrow (\Lambda^2 \odot \mathbf{X} \odot \mathbf{B}) \alpha_c$ 
  2:  $\mathbf{v} \leftarrow (\Lambda^2 (\text{diag}(\alpha_c) \alpha_c) + \varepsilon \mathbf{1}_n)$ 
  3: return  $\mathbf{u} \oslash \mathbf{v}$ 

func update_αc(X, B, αr, Λ)
  1:  $\mathbf{u} \leftarrow (\Lambda^2 \odot \mathbf{X} \odot \mathbf{B})^\top \alpha_r$ 
  2:  $\mathbf{v} \leftarrow (\Lambda^2)^\top (\text{diag}(\alpha_r) \alpha_r) + \varepsilon \mathbf{1}_m$ 
  3: return  $\mathbf{u} \oslash \mathbf{v}$ 

func update_B(W, {αr(k)}k=1K, {αc(k)}k=1K)
  1: for i = 1, ..., n do
  2:   for j = 1, ..., m do
  3:      $\mathbf{s} \leftarrow [(\alpha_r^{(k)})_i (\alpha_c^{(k)})_j]_{k=1}^K$ 
  4:      $\mathbf{b} \leftarrow \text{search}(W_{ij}, \mathbf{s})$ 
  5:     for k = 1, ..., K do
  6:        $(\mathbf{B}^{(k)})_{ij} \leftarrow \mathbf{b}_k$ 
  7:     end for
  8:   end for
  9: end for
  10: return  $\{\mathbf{B}^{(k)}\}_{k=1}^K$ 

func build_importance_mask(Z, λ)
  1:  $\mu \leftarrow \text{mean}(\mathbf{Z}), \quad \sigma \leftarrow \text{std}(\mathbf{Z})$ 
  2:  $\mathbf{M} \leftarrow \mathbb{I}(|\mathbf{Z} - \mu| > 3\sigma)$ 
  3: return  $\mathbf{1}_{n \times m} + (\lambda - 1) \mathbf{M}$ 

```

Quant-dLLM ABMP

- Redistribute the quantization budget at block level to enhance accuracy without increasing the overall memory used

Algorithm 3 ABMP: Adaptive Blockwise Mixed Precision

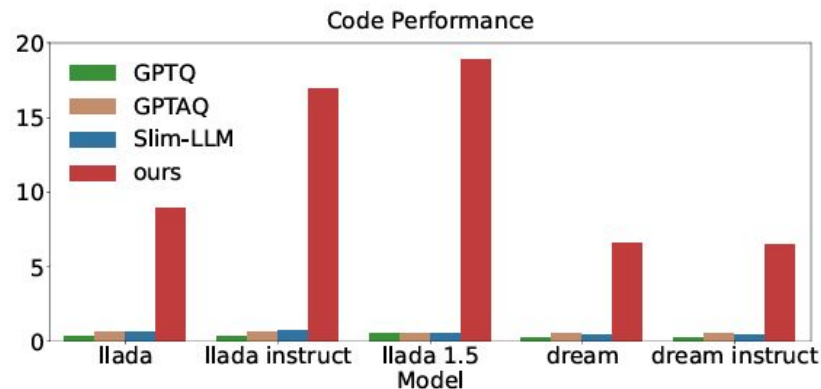
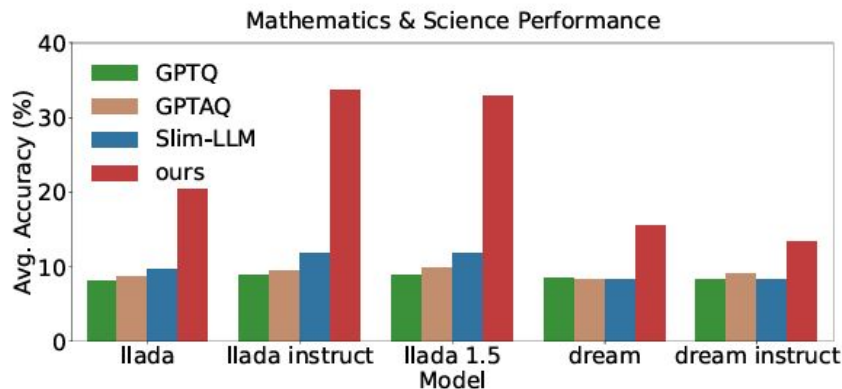
- 1: Given block partition $\mathcal{G}$ and importance $\mathbf{Z}$
 - 2: Compute $s_g = \sum_{(i,j) \in g} \mathbf{Z}_{ij}$ for all $g \in \mathcal{G}$
 - 3: Set $k \leftarrow \lfloor 0.05 |\mathcal{G}| \rfloor$ and sort blocks by s_g
 - 4: Assign $b_g = 3$ to top- k , $b_g = 1$ to bottom- k , others $b_g = 2$ ($\frac{1}{|\mathcal{G}|} \sum_g b_g = 2$)
 - 5: Apply DAQ quantization to each block using its assigned order $K = b_g$.
-

Quant-dLLM Experiments

- 128 calibration samples from C4 dataset with sequences of length 4096 tokens are used as calibration data.
- Quant-dLLM is evaluated on
 - General-knowledge QA: MMLU, ARC-E/C, HellaSwag, WinoGrande, PIQA, and BBH.
 - Mathematical and scientific reasoning: GSM8K, MATH, and GPQA.
 - Code generation: HumanEval and MBPP.
- Results are evaluated against GPTQ, GPTAQ, Slim-LLM
- Models used are Llama-8B base, Llama-8B-Instruct, Llama-1.5, Dream-7B-Base and Dream-7B-Instruct

Quant-dLLM Results

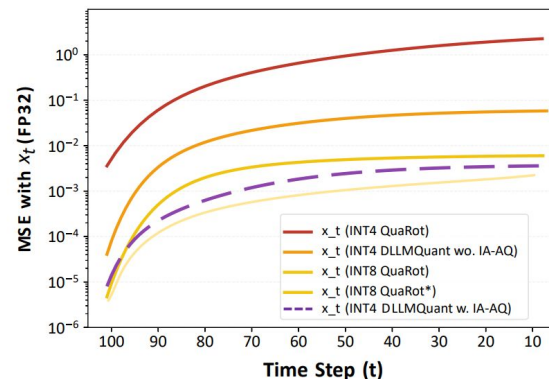
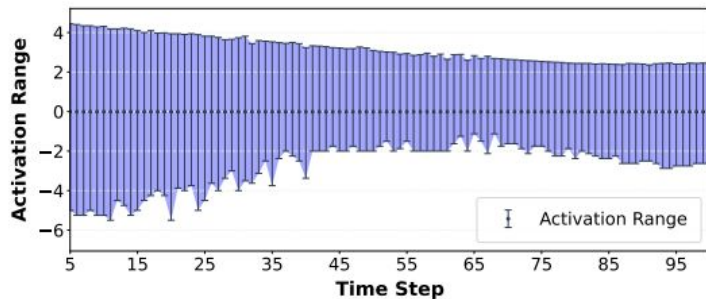
- Quant-dLLM has the highest average accuracy among 2-bit baselines on general, math and code tasks



dLLM Quant

Key issues in DLM quantization:

- Distinct token distributions are created across different decoding steps and these are not captured in current PTQ techniques
- Quantization errors accumulate across iterations
- Masked tokens remain probabilistic while unmasked tokens stabilize



dLLM Quant Framework

DLLMQuant includes three techniques:

1. Temporal mask adaptive sampling (TMAS): capturing distributions across timesteps in calibration data
2. Interaction-aware activation quantization (IA-AQ): allocate quantization resources and prevents accumulation of errors across timesteps
3. Certainty guided quantization (CGQ): using the mask status and token scores to include weighting in error compensation

dLLM Quant TMAS

- Existing calibration methods not work on DLMs due to iterative denoising and dynamic masking.
- To solve this sample inputs from different time steps that cover diverse masking ratios.

Algorithm 1: Temporal-Mask Adaptive Sampling (TMAS)

Require: Inputs $\mathcal{X}$, Block count B , Time steps T

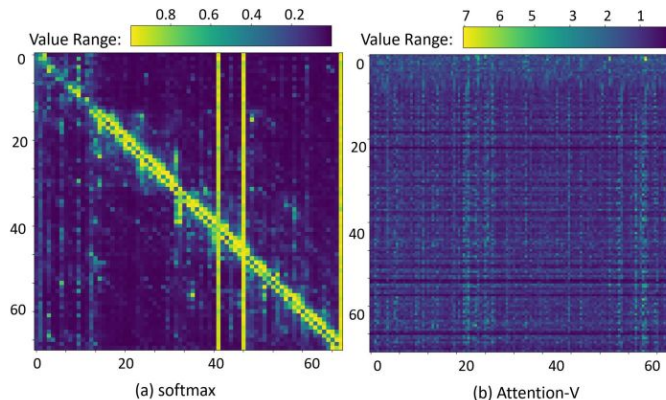
Ensure: Calibration dataset $\mathcal{D}_c$

```
1:  $s \leftarrow \lfloor T/B \rfloor$  {Steps per block}
2:  $n \leftarrow \lfloor 512/B \rfloor$  {Samples per block}
3:  $\mathbf{p} \leftarrow n \cdot [0.3, 0.2, 0.2, 0.3]$  {Target proportion of per mask ratio interval}
4:  $\mathbf{C} \leftarrow \text{zeros}(B, 4)$  {Sampling counter matrix}
5:  $\mathcal{D}_c \leftarrow \emptyset$  {Initialize calibration data}
6: Function ClassifyMaskRatio( $r$ ):
7:   return  $[0, 1, 2, 3][\lfloor (r \geq 0.2) + (r \geq 0.5) + (r \geq 0.8) \rfloor]$ 
8: for  $x \in \mathcal{X}$  do
9:   for  $t = T - 1$  to 0 do
10:     $y_t \leftarrow \text{Model}(x)$ 
11:     $r_t \leftarrow |y_t|_{\text{unmasked}} / |y_t|_{\text{total}}$ 
12:     $m \leftarrow \text{ClassifyMaskRatio}(r_t)$ 
13:     $\text{block} \leftarrow \lfloor t/s \rfloor$ 
14:    if  $\mathbf{C}[\text{block}, m] < \mathbf{p}[m]$  then
15:       $\mathcal{D}_c \leftarrow \mathcal{D}_c \cup \{x\}$ 
16:       $\mathbf{C}[\text{block}, m] \leftarrow \mathbf{C}[\text{block}, m] + 1$ 
17:    end if
18:  end for
19: end for
```

dLLM Quant IA-AQ

- The matmul between the output of softmax operation and value matrix in attention is a key factor for error accumulation in DLMs
- The softmax output exhibits notable sparsity in which the larger values are near the diagonal and only in some tokens while the rest are almost 0
- The value matrix has substantial variability across token and channel dimensions

$$\text{Softmax}(QK^T) \cdot V$$



dLLM Quant IA-AQ

- Use the softmax output as weighting term in the value matrix quantization error calculation

$$\mathcal{L}(s) = \left\| \left(\left\lfloor \frac{V - z}{s} \right\rfloor - V \right) \cdot \text{Deq}(O_{\text{softmax}}) \right\|_F^2$$

$$\hat{s} = (V_{\max} - V_{\min}) / (Q_{\max} - Q_{\min})$$

$$s = \alpha \odot \hat{s} = \arg \min_{\alpha \in \{1.0, 0.8\}} L(\alpha \odot \hat{s})$$

dLLM Quant CGQ

- Unmasked tokens and masked ones with low confidence scores do not propagate quantization errors
- Tokens with high output scores affect propagatio
- Place greater emphasis on the unmasked regions with higher scores during quantization through using weighted hessian matrices

$$H = (X \odot (\mathbf{1}[X_t = M] + \sqrt{sc_t})) \times (X \odot (\mathbf{1}[X_t = M] + \sqrt{sc_t}))^\top$$

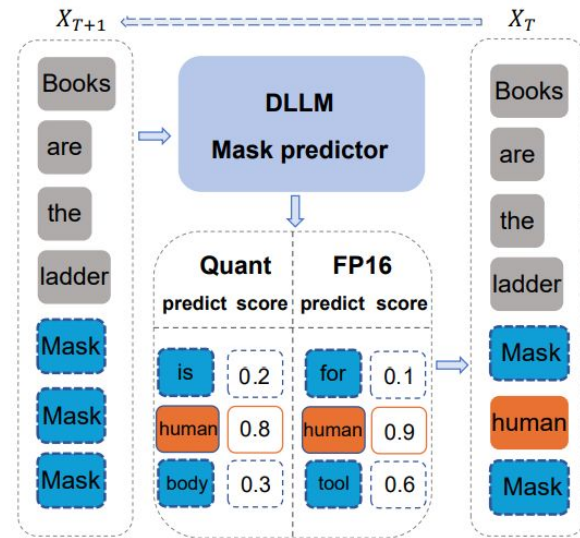


Figure 4: DLLMs iterative inference with masking and re-masking strategies. The quantization errors of unmasked tokens and masked tokens with low confidence scores do not affect the input of the next iteration.

dLLM Quant Results

Model	Method	Truth.	Arc.	Hella.	Wino.	PIQA	MMLU	C-EVAL	Hum.	GSM8K	Avg.
LLADA	FP	47.49	44.03	54.06	74.9	74.65	65.85	69.54	32.92	67.48	59.87
	RTN	40.45	41.83	45.40	64.72	67.95	49.26	57.95	14.02	16.56	44.23
	AWQ	40.87	42.92	46.14	66.88	69.43	51.22	58.43	20.10	36.88	48.09
	textDLLMQuant ⁺	41.53	43.44	46.51	67.87	70.12	51.72	59.38	22.13	40.66	49.26
	QuaRot	42.53	44.20	49.76	69.85	70.75	55.96	56.32	25.33	44.57	51.03
	textDLLMQuant ⁺⁺	43.53	44.18	51.00	71.85	73.94	57.77	61.22	28.92	56.25	54.29
LLADA-1.5	FP	47.2	88.5	74.7	74.8	74.86	66.0	70.05	49.4	83.3	69.86
	RTN	39.51	81.77	56.82	65.27	66.91	48.88	58.96	23.44	36.56	53.12
	AWQ	40.96	82.22	66.84	67.93	69.32	51.12	60.03	30.07	57.95	58.49
	textDLLMQuant ⁺	42.14	83.38	70.09	68.69	70.22	51.63	61.28	32.44	59.55	59.94
	QuaRot	43.21	84.23	65.34	69.55	70.17	56.23	57.66	37.33	65.86	61.06
	DLLMQuant ⁺⁺	43.87	84.18	69.20	71.74	73.58	57.27	60.04	44.58	74.33	64.31
DREAM	FP	49.76	59.80	73.30	74.50	75.66	69.5	64.89	57.9	77.2	66.94
	RTN	41.25	54.34	58.32	64.52	66.26	51.51	49.89	28.90	30.82	49.53
	AWQ	43.66	57.82	65.57	67.13	68.96	55.5	53.27	33.14	48.98	54.89
	textDLLMQuant ⁺	44.14	58.38	66.93	68.66	69.53	57.63	54.21	35.12	51.14	56.19
	QuaRot	47.58	58.18	67.13	70.05	70.36	69.5	53.19	34.48	59.20	58.85
	textDLLMQuant ⁺⁺	47.86	59.43	70.14	71.54	72.09	69.5	55.89	44.50	66.17	61.90

Table 1: Results of RTN, AWQ, QuaRot, and ours DLLMQuant with 4-bit weight and activation quantization among 9 tasks on LLADA-8B, LLADA-1.5-8B, DREAM-7B). DLLMQuant⁺ denotes DLLMQuant based on AWQ, and DLLMQuant⁺⁺ denotes DLLMQuant based on QuaRot.

dLLM Quant Results

MODEL	Speed (Tokens/s)			Memory (GB)		
	FP	Quant	Speed Up	FP	Quant	Mem. Sav.
LLADA	34.59	50.14	1.71	15.89	4.91	3.24
LLADA-1.5	35.55	60.43	1.70	15.88	4.90	3.24
DREAM	23.27	35.84	1.54	13.95	4.44	3.14

Table 4: Speedup and memory saving of three DLLMs, compared between our 4-bit implementation and FP16.

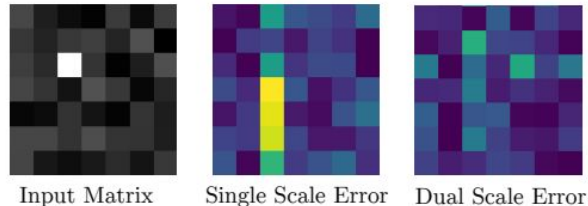
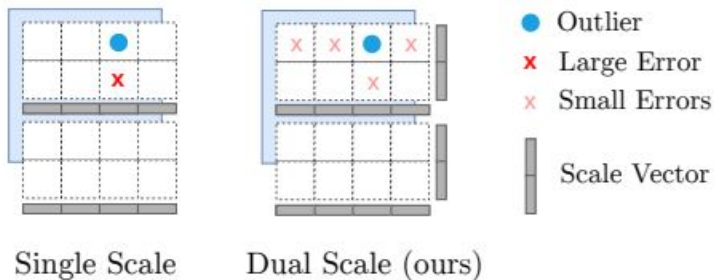
SINQ: Sinkhorn-Normalized Quantization for Calibration-Free Low-Precision LLM Weights

- SINQ is a model agnostic calibration free uniform quantization method
- Calibration can lead to overfitting and bias and adds extra compute time to quantization
- Non-uniform formats can improve quantization but add extra cost of lookups during inference and cannot be combined with activation quantization in compute limited scenarios

SINQ: Quantized Parametrization

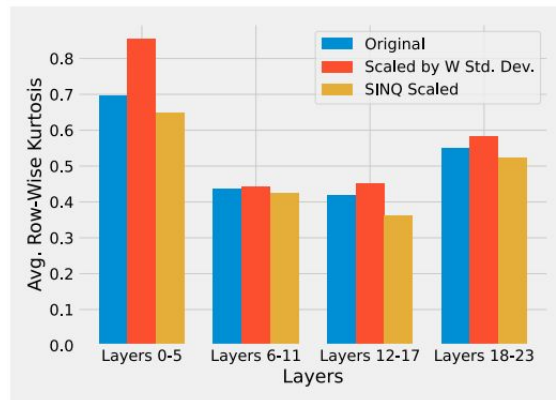
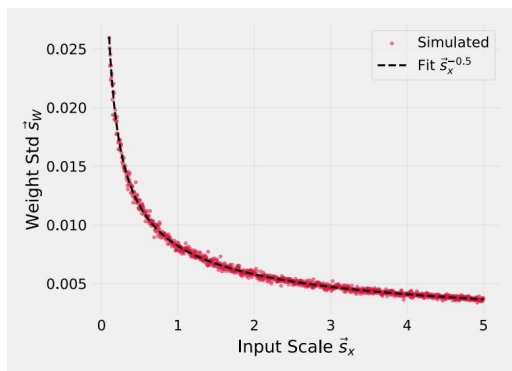
- Use scaling two vectors to trade off errors across rows and columns, reducing the overall approximation error
- Usually 1-dimensional column tiling is used and this paper uses the same thing but on dual scales so the number of parameters per tile increase by M which is negligible

$$2 \times N \times M/T$$
$$2 \times N \times M/T + M$$



SINQ: Representation Space

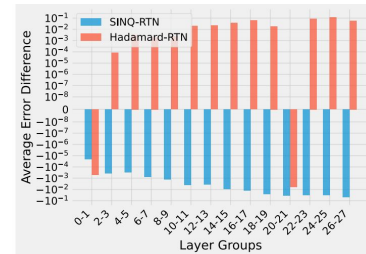
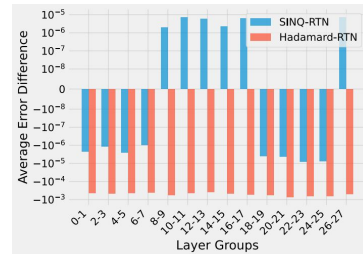
- Activation awareness does not require calibration data.
- There is a strong predictive power between the per-column weight standard deviation and the sample mean of the absolute value of the input to a given layer
- Scaling the input by inverse standard deviation of the weight matrix can achieve activation awareness



SINQ: Sinkhorn-Knopp Algorithm

- Balance between normalization of columns and rows to approximate activation aware quantization and keeping the row kurtosis low
- Minimize the matrix imbalance which leads to reduction in the activation reconstruction error of the output

$$I(\mathbf{W}) = \frac{\max(\max_i \sigma_i^{\text{row}}, \max_j \sigma_j^{\text{col}})}{\min(\min_i \sigma_i^{\text{row}}, \min_j \sigma_j^{\text{col}})}$$



Algorithm 1 SINQ: A dampened Log-Space Sinkhorn-inspired Normalization. Iteratively normalize the standard deviation of the rows and columns of the matrix to be quantized. Then apply a standard quantization method.

Require: Weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$, Iterations K , Bits b , Step-Sizes $s_{\min}, s_{\max}$

Ensure: Quantized weights $\mathbf{Q}$, Scales $\vec{s} \in \mathbb{R}^m, \vec{t} \in \mathbb{R}^n$

```

1:  $\vec{\sigma}_{\text{row}} \leftarrow \vec{\sigma}^{\text{row}}(\mathbf{W}); \vec{\sigma}_{\text{col}} \leftarrow \vec{\sigma}^{\text{col}}(\mathbf{W})$ 
2:  $\tau \leftarrow \min(\min(\vec{\sigma}_{\text{row}}), \min(\vec{\sigma}_{\text{col}}))$  {Target variance}
3:  $\mathbf{u} \leftarrow \vec{0}_m; \mathbf{v} \leftarrow \vec{0}_n$  {Initialize log-scales}
4:  $\theta^* \leftarrow \{\mathbf{u}, \mathbf{v}\}; I_{\text{best}} \leftarrow \infty$  {Track best solution}
5: for  $k \leftarrow 1$  to  $K$  do
6:    $\hat{\mathbf{W}} \leftarrow (\mathbf{W} \odot \exp(\mathbf{u})) \odot \exp(\mathbf{v})$  {Apply current scales}
7:    $I_{\text{curr}} \leftarrow \text{Imbalance}(\hat{\mathbf{W}})$ 
8:   if  $I_{\text{curr}} < I_{\text{best}}$  then
9:      $I_{\text{best}} \leftarrow I_{\text{curr}}; \theta^* \leftarrow \{\mathbf{u}, \mathbf{v}\}$  {Snapshot best scales}
10:  end if
11:   $\vec{\delta}_{\text{col}} \leftarrow \log(\text{clamp}(\vec{\sigma}^{\text{col}}(\hat{\mathbf{W}})/\tau, s_{\min}, s_{\max}))$ 
12:   $\vec{\delta}_{\text{row}} \leftarrow \log(\text{clamp}(\vec{\sigma}^{\text{row}}(\hat{\mathbf{W}})/\tau, s_{\min}, s_{\max}))$ 
13:   $\mathbf{v} \leftarrow \mathbf{v} + \vec{\delta}_{\text{col}}$  {Update col log-scales}
14:   $\mathbf{u} \leftarrow \mathbf{u} + \vec{\delta}_{\text{row}}$  {Update row log-scales}
15: end for
16:  $\vec{s}, \vec{t} \leftarrow \exp(\mathbf{u}^*), \exp(\mathbf{v}^*)$  {Recover best linear scales}
17:  $\hat{\mathbf{W}} \leftarrow (\mathbf{W} \odot \vec{s}) \odot \vec{t}$ 
18:  $\mathbf{Q}, \vec{z}, \vec{s}_q \leftarrow \text{RoundToNearest}(\hat{\mathbf{W}}, b)$ 
19: return  $\mathbf{Q}, \vec{z}, (\vec{s}_q \odot \vec{s}), \vec{t}$ 

```

SINQ Experiments

- Used 4 bit and 3 bit quantization methods, report perplexities for language modeling tasks and flip percentages for QA tasks.
- Uncalibrated non uniform quantization experiments were done by replacing RTN with NF4 quantizer
- Calibrated uniform quantization is done by combining SINQ with AWQ
- Experiments were done on Qwen3 models, Llama, deepseek and other MoE models.

SINQ Results

		Qwen3-14B				Qwen3-32B				
Method		<i>HellaSwag</i>	<i>PIQA</i>	<i>MMLU</i>	Avg. ↓	<i>HellaSwag</i>	<i>PIQA</i>	<i>MMLU</i>	Avg. ↓	
CALIBRATION-FREE	3-BIT	RTN [†]	8.44	8.60	10.97	9.34	22.84	17.08	10.61	16.84
		Hadamard + RTN [†]	10.68	10.93	16.21	12.60	19.83	13.17	12.81	15.27
		HQQ	7.99	7.94	14.28	10.07	7.23	9.30	10.98	9.17
		SINQ (ours)	5.34	7.02	10.82	7.73	5.54	7.13	10.21	7.63
	4-BIT	RTN [†]	2.92	4.57	4.89	4.13	4.18	6.31	5.28	5.26
		BnB (FP4)	4.21	5.71	6.72	5.55	12.32	9.14	6.25	9.24
		BnB (NF4)	2.66	3.10	4.70	3.49	3.73	3.48	4.76	3.99
		Hadamard + RTN [†]	3.63	5.55	4.88	4.69	4.01	6.02	5.32	5.12
		HQQ	2.81	4.35	5.17	4.11	5.83	5.18	4.98	5.33
		SINQ (ours)	2.36	3.37	4.65	3.46	2.52	3.59	4.69	3.60
CALIBRATED	3-BIT	GPTQ	5.18	7.83	11.17	8.06	6.33	8.76	10.25	8.45
		Hadamard [†] + GPTQ	5.14	7.56	11.15	7.95	5.52	8.71	10.08	8.10
		A-SINQ (ours)	5.13	7.18	10.36	7.56	5.23	7.62	10.15	7.67
	4-BIT	GPTQ	2.24	4.13	4.56	3.64	2.78	3.48	4.80	3.69
		Hadamard [†] + GPTQ	2.22	3.54	4.53	3.43	2.70	3.54	4.79	3.68
		AWQ	2.23	3.26	4.10	3.20	2.59	4.13	4.44	3.72
		A-SINQ (ours)	2.20	3.11	4.23	3.18	2.57	3.86	4.38	3.60

		Qwen3-1.7B			Qwen3-14B			Qwen3-32B		
Method		Mem.	Wiki2↓	C4↓	Mem.	Wiki2↓	C4↓	Mem.	Wiki2↓	C4↓
Original (BF16)		3.44	16.67	19.21	29.54	8.64	12.01	65.52	7.60	10.77
3-BIT	RTN [†]	1.28	32.43	31.10	9.23	10.50	14.88	17.61	30.78	35.83
	Hadamard + RTN [†]	1.28	32.40	31.07	9.23	10.60	15.10	17.61	11.26	14.83
	HQQ	1.28	32.10	30.54	9.23	10.73	14.39	17.62	9.09	12.58
	SINQ (ours)	1.28	22.39	24.88	9.25	9.33	12.90	17.61	8.79	11.83
4-BIT	RTN [†]	1.42	18.74	20.81	10.54	8.95	12.50	20.78	8.92	12.80
	Hadamard + RTN [†]	1.42	19.10	20.70	10.54	8.85	12.35	20.78	8.28	11.60
	HQQ	1.42	18.96	22.10	10.54	8.78	12.36	20.78	8.62	12.20
	SINQ (ours)	1.42	17.14	19.83	10.56	8.76	12.21	20.73	7.74	10.96

SINQ Results

Table 10. Average quantization time (seconds) across 10 runs for some Qwen3 models on GPU, comparing different quantization methods. The rightmost column reports the relative average slowdown with respect to RTN.

Method	Qwen3-1.7B	Qwen3-4B	Qwen3-8B	Qwen3-14B	Qwen3-32B	Avg. cost
<i>RTN</i>	<i>2.91 s ± 0.11</i>	<i>6.32 s ± 0.06</i>	<i>11.35 s ± 0.31</i>	<i>20.61 s ± 0.87</i>	<i>46.79 s ± 2.52</i>	<i>1.00$\times$</i>
HQQ	3.65 s ± 0.13	10.15 s ± 0.27	24.06 s ± 1.54	43.62 s ± 0.54	122.45 s ± 2.45	2.32 $\times$
GPTQ	193.33 s ± 1.68	426.89 s ± 0.75	669.06 s ± 0.84	1160.37 s ± 1.68	3064.62 s ± 24.33	62.68 $\times$
AWQ	104.63 s ± 9.26	225.27 s ± 3.91	392.51 s ± 2.86	695.29 s ± 1.19	1613.75 s ± 9.79	34.46 $\times$
A-SINQ (<i>ours</i>)	23.86 s ± 0.17	49.81 s ± 0.13	92.17 s ± 0.33	173.93 s ± 0.38	411.95 s ± 0.57	8.54 $\times$
SINQ (<i>ours</i>)	3.03 s ± 0.29	6.33 s ± 0.52	13.23 s ± 0.64	21.38 s ± 2.15	51.56 s ± 2.00	1.09$\times$

Key Takeaways

- DLMS dynamic masking iterative generation of text and bidirectional attention makes quantization more difficult
- Quantization of the matrix multiplication following the softmax operation in attention is primary source of error propagation in dLLMs
- Uncalibrated uniform quantization methods like SINQ can offer several advantages and perform even better than calibrated and uniform quantizers

Plan for next week:

- Apply SINQ model on Llada 8B/ Dream 7B models and evaluate the results on some benchmarks